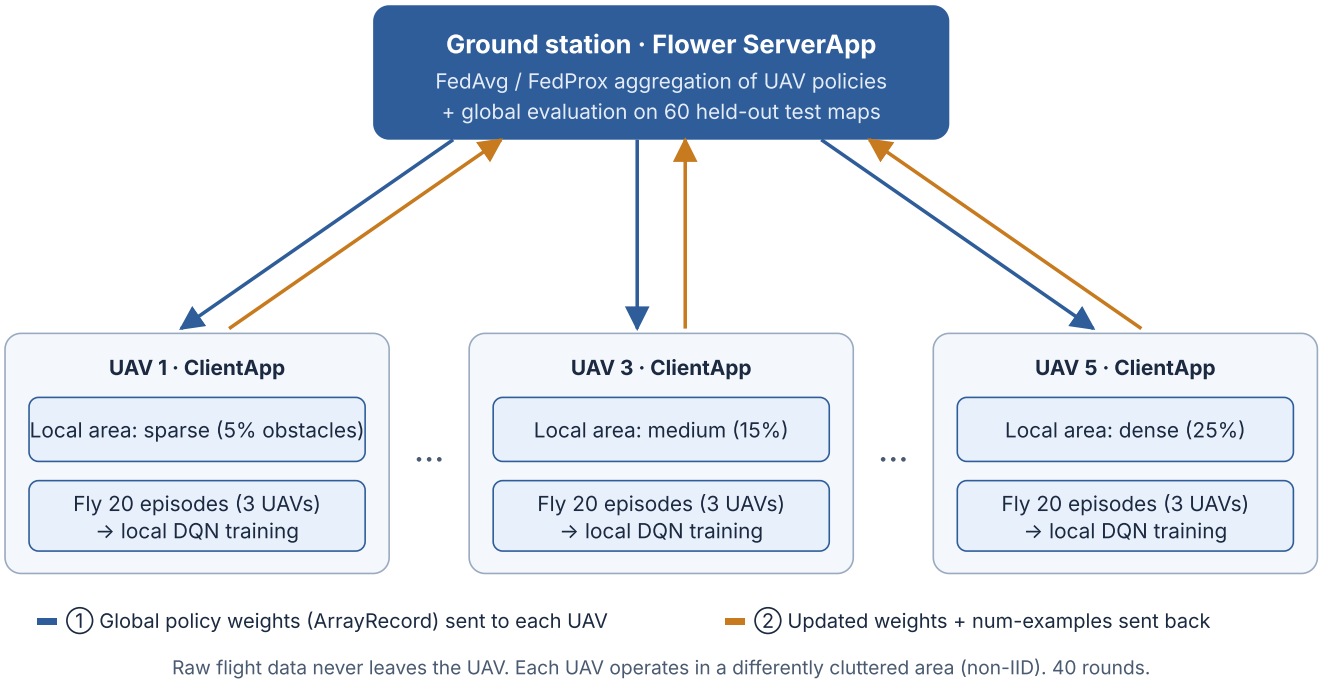
Progress report · Distributed Analytics for Industrial Automation (IM60210) · Ayan Ansari · 3 Nov 2026

Distributed Path Planning for Multi-UAV Systems using Federated Reinforcement Learning (Flower)

Idea in one paragraph

Several UAVs learn **one shared path-planning policy** without sending their raw flight data anywhere. Each UAV is a **Flower ClientApp**: it flies practice missions in its own area and trains a small deep Q-network (DQN) locally. Only the **model weights** go to a ground-station **ServerApp**, which averages them with **FedAvg** (or FedProx) and sends the improved policy back. When flying, each UAV plans its own moves from a **local 7×7 view**, with no central planner, avoiding obstacles and other UAVs.

System architecture



Raw flight data never leaves the UAV. Each UAV operates in a differently cluttered area (non-IID). 40 rounds.

95%	+14 pts	≈ 7×	5 × 40
UAVs reach goal (FedAvg)	vs UAVs learning alone	less data sent than pooling raw experience	UAV clients × federated rounds

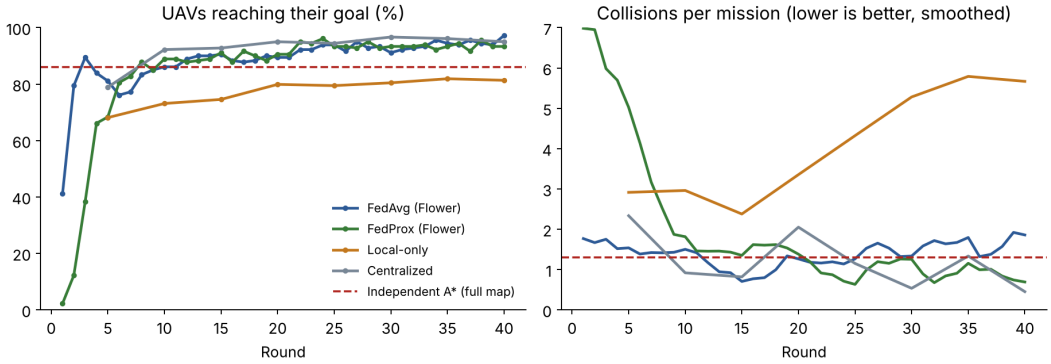
Done so far

- ✔ Multi-UAV grid-world simulator: obstacles, unique starts and goals, UAV–UAV and UAV–obstacle collision handling.
- ✔ Classical baseline: **A*** per UAV with full map knowledge.
- ✔ DQN agent with a shared policy across all UAVs (Double DQN, the global model is the target network).
- ✔ **Flower app** (`ClientApp` + `ServerApp`, same API as the course's Flower–PyTorch tutorial), run with `flwr run .` on the Simulation Runtime.
- ✔ Non-IID setup: each UAV trains in a different obstacle density (5–25%).
- ✔ Experiments: FedAvg, FedProx, local-only and centralized, plus a communication-cost estimate and a demo GIF.

Experimental setup

Item	Value
World	12×12 grid, random obstacles (5–25%), 3 UAVs per mission, each with its own start and goal
What a UAV sees	Local 7×7 window: obstacles, other UAVs, its goal, cells it has already visited; plus goal direction
Actions / reward	stay, N, S, W, E · +2 reaching goal, −0.5 per collision, −0.05 per step, + progress toward goal
Model	MLP 199 → 128 → 128 → 5 (42,757 parameters, 0.17 MB)
Federation	5 UAV clients, 40 rounds; per round each client flies 20 episodes and does 500 gradient steps
Test	60 held-out missions (unseen maps, mixed densities), same for every method

Results



Evaluation on the 60 test missions after every federated round. Local-only and centralized are evaluated every 5 rounds.

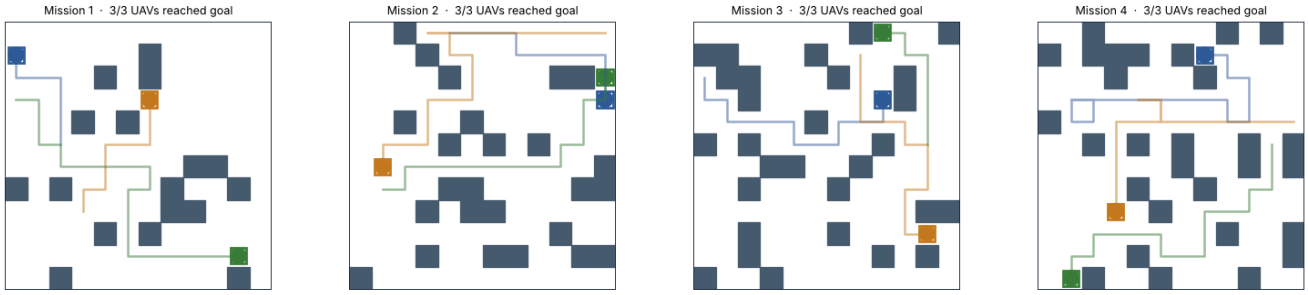
Method	UAVs reaching goal	Whole mission solved	Collisions / mission	Path length ÷ optimal
Independent A* (full map, no coordination)	86.1%	80.0%	1.30	1.00
Local-only (each UAV learns alone)	81.2%	59.2%	5.58	1.35
FedAvg (Flower)	**95.2%**	**87.2%**	**2.15**	**1.22**
FedProx, μ = 0.01 (Flower)	**94.1%**	**85.0%**	**0.84**	**1.18**
Centralized (all raw data pooled; upper bound)	95.9%	89.4%	0.77	1.16

Average of the last 3 evaluation points; one random seed. A* is planned separately for each UAV and run open-loop, so it does not avoid other UAVs.

Key findings so far

- **Federation clearly helps:** FedAvg beats UAVs learning alone by 14 points (95% vs 81%) and solves far more full missions (87% vs 59%).
- **Close to centralized** (95.2% vs 95.9%) while sending **≈ 68 MB** of weights instead of **≈ 480 MB** of raw flight data, and the data stays on the UAVs.
- **Better than uncoordinated A*** at getting UAVs home, because the learned policy reacts to other UAVs. Its paths are about 20% longer, since it only sees a 7×7 window.
- **FedProx** gives similar success with fewer collisions than FedAvg under non-IID areas. This needs multiple seeds to confirm.

Demo: shared Flower policy on unseen test missions



Three UAVs (blue, orange, green), each flying to its own goal (square) on maps never seen in training. Lines show the paths flown. An animated version is in `figures/uav_demo.gif`.

Limitations (honest status)

- Results come from **one random seed**; reinforcement learning is noisy, so final numbers need 3 or more seeds with error bars.
- Small 2D grid world, 3 UAVs per mission, and simulation only (Flower Simulation Runtime on one machine).
- The A* baseline does not coordinate between UAVs. A fair classical comparison needs a coordinated planner (prioritized or space-time A*).
- Communication cost is an estimate (model size × transfers vs replay-data size).

Next steps

- ☐ Repeat all methods with 3 or more seeds; report mean ± std and worst-UAV success.
- ☐ Scalability: 5 → 10 → 20 UAV clients; larger maps (20×20); more UAVs per mission.
- ☐ Robustness: UAV dropout (`fraction-train` < 1) and stronger non-IID splits.
- ☐ Coordinated A* baseline (prioritized planning) for a fair comparison.
- ☐ **Real distribution:** Flower Deployment Runtime with a SuperLink plus one SuperNode per UAV in separate containers, measuring round time and bandwidth.
- ☐ Optional: differential privacy (Flower's DP strategies) and the privacy vs performance trade-off.

Code

`uavfl/env.py`	Multi-UAV grid world
`uavfl/task.py`	Q-network, experience collection, DQN training, evaluation
`uavfl/client_app.py`	Flower ClientApp: one per UAV (`@app.train`, `@app.evaluate`)
`uavfl/server_app.py`	Flower ServerApp: FedAvg / FedProx, global evaluation
`experiments/`	Baselines, plots, communication cost, demo GIF

Run: `pip install -e .` → `flwr run . --stream` (FedAvg) · `flwr run . --run-config 'prox-mu=0.01 …'` (FedProx)